

COMP20007 Assignment 2 Task 1C: Written Analysis

Part A: Dynamic Programming Solution

1. State Representation

Each state in the dynamic programming solution is represented by $dp[lake][time]$, where:

- $lake \in \{0, 1, \dots, N-1\}$ represents the current lake ID
- $time \in \{0, 1, \dots, C\}$ represents the remaining time until breeding season

State Value: $dp[lake][time]$ stores the maximum fat achievable when at $lake$ with $time$ days remaining, or -1 if the state is unreachable.

Constraint Management:

- **Fat bounds:** All transitions ensure fat remains in $(0, F]$, where transitions resulting in $fat \leq 0$ are rejected, and $fat > F$ is capped at F
- **Time constraint:** Transitions are only valid if sufficient time remains ($time \geq travel_cost$)
- **Survivability:** Transitions require $current_fat > travel_cost$ to survive the journey

Auxiliary structures:

- $parent[lake][time]$ and $parentTime[lake][time]$ store backtracking information for path reconstruction

2. Recurrence

Base Case:

```
dp[S][C] = K if K > 0, otherwise -1
All other dp[lake][time] = -1 initially
```

Recurrence Relation: For each state $dp[u][t]$ where $dp[u][t] > 0$, and for each outgoing edge (u,v) with cost w :

```
if t ≥ 1 and dp[u][t] > w:
    fat_after_travel = dp[u][t] - w
    fat_after_eating = fat_after_travel + gain[v]

    if fat_after_eating > 0:
        fat_final = min(fat_after_eating, F)
        if dp[v][t-1] < fat_final:
            dp[v][t-1] = fat_final
            parent[v][t-1] = u
            parentTime[v][t-1] = t
```

Correctness Justification:

- **Optimality:** The recurrence explores all valid transitions and keeps only the maximum fat for each state
- **Feasibility:** All constraints (fat survival, time limits, maximum capacity) are enforced during transitions
- **Completeness:** Processing in reverse time order (C down to 1) ensures all dependencies are resolved before being used

3. Time Complexity**Variables:**

- N = number of lakes
- C = time limit
- M = number of rivers (total edges)
- Let E = total number of outgoing edges across all lakes = M

Time Complexity: $O(C \times E) = O(C \times M)$

- Outer loop: C iterations (time steps)
- Middle loop: N iterations (lakes), but only processes reachable states
- Inner loop: Variable per lake based on out-degree, totaling E operations across all lakes per time step

Space Complexity: $O(N \times C)$

- Three 2D arrays: dp, parent, parentTime, each of size $N \times (C+1)$

Note: While the naive bound is $O(N^2C)$ (assuming each lake connects to all others), the actual complexity is $O(CM)$ where M is the number of edges in the input graph.

4. Backtrace

Final Answer Location: The optimal solution is found by examining all `dp[0][t]` for destination lake O across all time values $t \in \{0, 1, \dots, C\}$:

```
bestFat = max{dp[0][t] : t ∈ {0,1,...,C}, dp[0][t] ≠ -1}
bestTime = min{t : dp[0][t] = bestFat} // Prefer earlier arrival
```

Path Reconstruction: Starting from state (O, bestTime), use parent pointers to trace backwards:

1. Initialize: current = O, time = bestTime
2. While current ≠ -1:
 - Add current to path
 - prev = parent[current][time]
 - time = parentTime[current][time]
 - current = prev
3. Reverse the collected path to get forward direction

Part B: Modifications for Variable Travel Time

Key Difference

In Part B, **travel time equals the fat cost** of traversing a river, rather than being fixed at 1 day.

Modified Elements

Recurrence (Updated): The transition condition changes from $t \geq 1$ to $t \geq w$, and the next state becomes $t - w$:

```
if t ≥ w and dp[u][t] > w:
    fat_after_travel = dp[u][t] - w
    fat_after_eating = fat_after_travel + gain[v]

    if fat_after_eating > 0:
        fat_final = min(fat_after_eating, F)
        if dp[v][t-w] < fat_final: // Note: t-w instead of t-1
            dp[v][t-w] = fat_final
            parent[v][t-w] = u
            parentTime[v][t-w] = t
```

Time Complexity: Remains $O(C \times M)$ as the total number of state transitions is bounded by the same factors, though individual transitions may consume more time units.

Unchanged Elements

- State representation remains $dp[lake][time]$
- Base case and constraint management unchanged
- Space complexity remains $O(N \times C)$
- Backtrace procedure identical

Implementation Notes

The provided code correctly implements both variants using a unified approach where $timeCost = (part == PART_A) ? 1 : travelCost$, demonstrating the minimal modification required between the two parts while maintaining the same algorithmic structure and complexity bounds.